

fhir-k8s-gitops: step-by-step guide

A tiny Node API that wraps the public HAPI FHIR test server, built into a Docker image by GitHub Actions, stored in GHCR, and deployed to Kubernetes (KillerCoda) by ArgoCD from Git.

Flow: code in Git → GitHub Actions builds image → GHCR stores it → YAML in Git names the image → ArgoCD makes the cluster match Git.

Repo: `github.com/rprrr736/fhir-k8s-gitops` (public) · **Image:** `ghcr.io/rprrr736/fhir-k8s-gitops:<tag>`
· **Laptop folder:** `C:\Ravi\Ravi\AI learning\fhir-k8s-gitops`

Commands marked **PowerShell** run on your laptop. Commands marked **KillerCoda** run in the playground terminal.

Phase A: Local app

1. Create the project folder and check tools

Makes the working folder and confirms Node, npm and Git exist. Docker is not needed on the laptop, because GitHub builds the image.

```
cd "C:\Ravi\Ravi\AI learning"
mkdir fhir-k8s-gitops
cd fhir-k8s-gitops
node -v
npm -v
git --version
```

2. Initialize Node and install Express

Creates `package.json` and installs the web framework.

```
npm init -y
npm install express
```

3. Write the first version of the API

Two endpoints: `/health` (am I alive?) and `/hello` (does my app answer?). Create the file with `code` `index.js`, paste, save with `Ctrl+S`.

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/health", (req, res) => res.json({ status: "ok" }));
app.get("/hello", (req, res) => res.json({ message: "Hello from Kubernetes" }));

app.listen(PORT, () => console.log(`API listening on port ${PORT}`));
```

Test it, then open `http://localhost:3000/hello` in a browser and stop with `Ctrl+C`:

```
node index.js
```

4. Add .gitignore and Dockerfile

`.gitignore` keeps `node_modules` and VS Code settings out of Git. The `Dockerfile` is the recipe for the image (the file has no extension).

`.gitignore`:

```
node_modules
.vscode
```

`Dockerfile`:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Phase B: GitHub and automatic image build

5. Create the GitHub repo (browser)

New repository named `fhir-k8s-gitops`, **Public**, with no README, `.gitignore` or license. Public lets ArgoCD read it without credentials.

6. First commit and push

Sets a repo-local identity so this repo uses the `rprrr736` account, then pushes. A sign-in window may appear.

```
git init -b main
git config user.name "rprrr736"
git config user.email "<your rprrr736 email>"
git add .
git status
git commit -m "Initial commit: tiny Node API and Dockerfile"
git remote add origin https://rprrr736@github.com/rprrr736/fhir-k8s-gitops.git
git push -u origin main
```

7. Add the GitHub Actions workflow

On every push to `main` (except changes under `k8s/`), GitHub builds the `Dockerfile` and pushes the image to GHCR. No secrets needed, because the built-in `GITHUB_TOKEN` logs in.

```

mkdir .github\workflows
code .github\workflows\build.yml

name: Build and push image

on:
  push:
    branches: [main]
    paths-ignore:
      - "k8s/**"
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/rprrr736/fhir-k8s-gitops:v1

git add .
git commit -m "Add GitHub Actions workflow"
git push

```

Watch the **Actions** tab until the run is green.

8. Make the image public (browser)

New GHCR packages are private, so the cluster could not pull them. Profile → **Packages** → fhir-k8s-gitops → **Package settings** → **Danger Zone** → **Change visibility** → **Public**.

Phase C: Kubernetes YAML

9. Write the Deployment and Service

The Deployment says "run this image on port 3000 and check /health ". The Service gives it a stable address on node port 30080.

```
mkdir k8s
code k8s\deployment.yaml
code k8s\service.yaml
```

k8s/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fhir-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: fhir-api
  template:
    metadata:
      labels:
        app: fhir-api
    spec:
      containers:
        - name: fhir-api
          image: ghcr.io/rprrr736/fhir-k8s-gitops:v1
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 3
            periodSeconds: 5
```

k8s/service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: fhir-api
spec:
  type: NodePort
  selector:
    app: fhir-api
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

```
git add .  
git commit -m "Add Kubernetes deployment and service"  
git push
```

Phase D: First deploy by hand (KillerCoda)

10. Open a Kubernetes playground

killercode.com → Playgrounds → **Kubernetes**. Sessions are temporary (about an hour), which is why everything lives in Git.

```
kubectl get nodes
```

11. Deploy with kubectl apply

Proves the image and YAML work before adding ArgoCD.

```
git clone https://github.com/rprrr736/fhir-k8s-gitops.git  
cd fhir-k8s-gitops  
kubectl apply -f k8s/  
kubectl get pods
```

Repeat get pods until Running and 1/1. ImagePullBackOff usually means the package is still private or the tag is wrong.

12. Call the API from the terminal

```
kubectl get svc fhir-api  
curl http://localhost:30080/health  
curl http://localhost:30080/hello
```

13. Call the API from Postman (optional)

Postman runs on your laptop, so it needs a public URL. In the KillerCoda page open **Traffic / Ports**, enter **30080**, and open it. Add the path yourself: <url>/hello. A bare URL shows "Cannot GET /", which only means your app answered but has no / route. Use a GET request in Postman, and keep the base URL in a variable, because each session gets a new one.

Phase E: ArgoCD

14. Install ArgoCD in the cluster

--server-side avoids an "annotation too long" error.

```
kubectl create namespace argocd  
kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj  
kubectl get pods -n argocd
```

Repeat until all seven pods are Running (1 to 3 minutes).

15. Write the ArgoCD Application (laptop)

One object that says "watch the k8s folder on main and deploy it to this cluster, automatically".

```
mkdir argocd
code argocd\application.yaml

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fhir-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/rprrr736/fhir-k8s-gitops.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

git add .
git commit -m "Add ArgoCD application"
git push
```

16. Apply the Application in the cluster

```
cd ~/fhir-k8s-gitops
git pull
kubectl apply -f argocd/application.yaml
sleep 30
kubectl get applications -n argocd
```

You want Synced and Healthy. From now on ArgoCD deploys; you no longer run `kubectl apply -f k8s/`.

Phase F: Making changes through Git

17. Scale by changing Git

In `k8s/deployment.yaml` change `replicas: 1` to `replicas: 2`, save, then:

```
Select-String -Path k8s\deployment.yaml -Pattern "replicas"
git add .
git commit -m "Scale to 2 replicas"
git push
```

```
kubectl get pods
```

A second pod appears with no `kubectl` from you (ArgoCD polls about every 3 minutes).

18. The release loop (used for every new version)

Replace `vN` with the new tag (`v2`, `v3` ...). **Never reuse a tag**, because Kubernetes caches images by tag.

1. Change the code in `index.js`, and change the last line of `build.yml` to `...:vN`. Save both, then verify they are really saved:

```
Select-String -Path index.js -Pattern "message"
Select-String -Path .github\workflows\build.yml -Pattern "tags"
git status
```

2. Commit and push, then wait for a **green** run in the Actions tab:

```
git add .
git commit -m "Build vN image"
git push
```

3. Only after it is green, change the image line in `k8s\deployment.yaml` to `:vN`, then verify, commit and push:

```
Select-String -Path k8s\deployment.yaml -Pattern "image:"
git add .
git commit -m "Deploy vN image"
git push
```

4. In KillerCoda, ask ArgoCD to check now instead of waiting, then verify:

```
kubectl annotate application fhir-api -n argocd argocd.argoproj.io/refresh=hard --overwrite
sleep 30
kubectl get pods
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
curl http://localhost:30080/hello
```

Phase G: FHIR routes

Use **fake test data only**. HAPI is a shared public server that anyone can read and change. Only delete patients you created.

19. Check the cluster can reach HAPI

```
curl -s -m 30 -H "Accept: application/fhir+json" "https://hapi.fhir.org/baseR4/Patient?_count=1" | he
```

You should see the start of a JSON Bundle .

20. Add the read routes

GET /patients (searches, optional ?name=) and GET /patients/:id . They call HAPI with Node's built-in fetch and return a trimmed patient (id , name , gender , birthDate). Locally, --use-system-ca is needed on the office network:

```
node --use-system-ca index.js
```

Open <http://localhost:3000/patients> , then <http://localhost:3000/patients/<id>> .

21. Add the write routes

POST /patients (create), PUT /patients/:id (replace the whole patient, so send all four fields) and DELETE /patients/:id . The complete final index.js is in the repo. It also uses app.use(express.json()) so request bodies are read.

22. Test locally in Postman

Start the app, then use these requests against <http://localhost:3000> in this order:

Request	URL	Expect
POST	/patients	201 and a new id
GET	/patients/<id>	200, same patient
PUT	/patients/<id>	200, changed name
DELETE	/patients/<id>	200 {"deleted": "<id>"}
GET	/patients/<id>	an error such as 404 or 410

For POST and PUT use Body → raw → JSON:

```
{
  "given": "Test",
  "family": "Demo",
  "gender": "female",
  "birthDate": "1990-01-01"
}
```

23. Ship the routes with the release loop

Run step 18 with v5 . (/hello was changed to say v5, so you can see the rollout.)

Phase H: Rebuilding after KillerCoda resets, and testing the pod

24. Rebuild the cluster from Git

When the playground expires, nothing is lost. In a new Kubernetes playground:

```
git clone https://github.com/rprrr736/fhir-k8s-gitops.git
cd fhir-k8s-gitops
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj
kubectl get pods -n argocd
```

When all seven pods are Running:

```
kubectl apply -f argocd/application.yaml
sleep 30
kubectl get applications -n argocd
kubectl get pods
```

ArgoCD creates the Deployment and Service from Git, with no `kubectl apply -f k8s/`.

25. Test the pod from Postman

Open **Traffic / Ports** for port **30080** to get the new session's URL, set it as the base URL, and repeat the table from step 22 against it (**POST** , **GET** , **PUT** , **DELETE** on `/patients`). The request path is Postman → KillerCoda URL → your pod → HAPI.

Lessons learned

- **Check saves before committing.** Run `Select-String` and `git status`. An unsaved edit once produced a v2 image with old code.
- **Never reuse an image tag.** Use a new tag for every change.
- **Build first, deploy second.** Wait for the green Actions run before changing `deployment.yaml`.
- **Two triggers:** a code push triggers the build, and a YAML push triggers the deploy. `paths-ignore: k8s/**` stops YAML edits from rebuilding.
- **ArgoCD polls about every 3 minutes.** Use the `refresh=hard` annotation to check right away.
- **"Cannot GET /"** means your app answered, and the URL just needs `/hello` or `/patients`.
- **Docker Hub only stores images.** Something with Docker still has to build them, and here GitHub Actions does it.
- **apiVersion in YAML is the Kubernetes API version.** It is unrelated to your image tag.

Ideas for later

- Automate the tag edit in `deployment.yaml` (GitHub Actions step, or Jenkins).
- Self-heal demo: delete the Service with `kubectl` and watch ArgoCD restore it.
- ArgoCD web UI.
- Add a React page, SMART/OAuth login, or the AI agent and RAG on top.

